

IMPERIAL

Imperial College of Science, Technology and Medicine

FPGA Acceleration of the AES Encryption Algorithm using Intel oneAPI SYCL

Name:	Mihir Koka
CID:	06067948
Date of submission:	27th April 2026

Research Highlight

This study investigates the acceleration of the Advanced Encryption Standard (AES) using FPGA-oriented implementation techniques in Intel oneAPI SYCL. AES is a natural target for hardware acceleration because it applies a repeated and regular round structure to 128-bit blocks (National Institute of Standards and Technology 2001). Prior FPGA AES work has shown that high throughput depends strongly on pipelining, initiation interval, and memory partitioning rather than on clock frequency alone (Manjith and Ramasubramanian 2018; Soltani and Sharifian 2015). This report therefore studies AES as a hardware design problem: the goal is not only to make AES run, but to understand which source-level transformations produce a more efficient FPGA schedule.

The project targets a CTR-style AES workload, where independent counter blocks allow block-level parallelism. The implementation covers the full AES round structure, including **SubBytes**, **ShiftRows**, **MixColumns**, and **AddRoundKey**. In addition, the **SubBytes** + **ShiftRows** stage is isolated as a detailed case study because it exposes several important FPGA design issues in a compact form: S-box lookup, byte permutation, memory access ordering, local buffering, and loop scheduling.

The main parts of the work can be described as:

- a full-round structural FPGA compile test integrating the four AES round operations in a CTR-style pipeline
- a comparison of full-pipeline optimisation methods, including deep pipelining, S-box banking, buffering and decoupling, non-aliasing constraints, and loop-independence restructuring
- a focused optimisation study of the **SubBytes** + **ShiftRows** stage, comparing a naive baseline, inner-loop unrolling, direct fusion, and buffered fused load-compute-store restructuring
- Analysis using initiation interval (II), block latency, estimated clock frequency, resource usage, memory-dependency bottlenecks, and compiler handshaking status
- a reproducible oneAPI/CMake workflow separating correctness checks with `fpga.emu` from FPGA-oriented design evaluation using `make report`.

1 Introduction

The Advanced Encryption Standard (AES) is the current U.S. federal standard for symmetric-key encryption and is widely used in secure storage, network communication, and cloud infrastructure (National Institute of Standards and Technology 2001). It operates on 128-bit blocks and applies a repeated sequence of transformations controlled by round-key material. This regular structure makes AES a strong candidate for hardware acceleration: in many practical settings, the important requirement is not merely the latency of one encrypted block, but the sustained throughput when many blocks are processed.

Dedicated hardware accelerators are often used when encryption becomes a throughput bottleneck for a general-purpose processor. CPU instruction-set extensions such as AES-NI provide an important software baseline (AbdAllah et al. 2020), but Manjith and Ramasubramanian

argue that FPGA-based AES designs can exploit parallelism, deep pipelining, and memory partitioning to improve throughput and resource efficiency (Manjith and Ramasubramanian 2018). Their AES accelerator work is particularly relevant to this project because it treats latency, initiation interval, and memory access structure as the primary design quantities. This report follows the same general motivation, but studies the problem through the Intel oneAPI SYCL high-level synthesis flow rather than a hand-written RTL flow.

FPGAs are useful for this investigation because the compilation flow exposes hardware-facing metrics that are not visible from a normal software runtime: pipeline structure, scheduled II, block estimated exit II, estimated clock frequency, resource utilisation, and memory-related bottlenecks (Intel 2024a; Intel 2023c). These metrics make it possible to distinguish between transformations that only improve emulator execution and transformations that actually improve the hardware schedule.

This report has two main goals. First, to study how the AES round structure can be mapped into a CTR-style FPGA pipeline. Second, to evaluate which implementation choices improve the hardware schedule and resource trade-off. The full-pipeline study examines deep pipelining, S-box banking, decoupling, non-aliasing, and loop-independence transformations. The stage-level study then examines **SubBytes** + **ShiftRows** in greater detail, showing why aggressive unrolling and direct fusion can actually worsen the FPGA report even when they appear faster under emulation.

2 Background and Scope

AES transforms a 128-bit state, commonly represented as a 4×4 matrix of bytes. AES-128, AES-192, and AES-256 use 10, 12, and 14 rounds respectively (National Institute of Standards and Technology 2001). Each standard round contains four core transformations:

1. **SubBytes**: a nonlinear byte substitution using a fixed S-box
2. **ShiftRows**: a cyclic row permutation of the state
3. **MixColumns**: a finite-field linear transformation applied to each column
4. **AddRoundKey**: a bitwise XOR between the state and round-key material.

The full implementation considers all four operations. At system level, the report studies how these operations can be staged, pipelined, buffered, and made independent to improve throughput. At stage level, the report focuses most closely on **SubBytes** + **ShiftRows**. This stage is technically useful as a case study because **SubBytes** creates lookup pressure through repeated S-box access, while **ShiftRows** changes the byte ordering and therefore the output memory pattern.

AES can be used in several modes of operation, including ECB, CBC, CTR, and GCM (Dworkin 2001). CTR mode is especially relevant here because each counter block can be encrypted independently before being XORed with the plaintext. This avoids the direct inter-block dependency present in CBC encryption and makes CTR a natural mode for studying block-level parallelism. Therefore we use a CTR-style workload to examine how independent AES blocks can be scheduled by the FPGA compiler.

3 Methodology and Metrics

The implementation was developed in SYCL using the Intel oneAPI FPGA flow. Kernels were written using the single-work-item model, which is designed for loop-based FPGA pipelines and exposes loop-carried dependencies directly to the compiler report (Intel 2024b). Each design was evaluated through two routes. The `fpga_emu` target was used to check functionality and provide a rough emulator runtime. For the isolated stage kernels, this runtime was measured using SYCL event profiling around the kernel submission, by subtracting the reported command start time from the command end time. This is useful for comparing emulator runs during development, but it is not treated as real FPGA throughput. The optimisation report generated by `make report` was used as the main FPGA-facing evidence, because it reports loop scheduling, II, estimated clock frequency, resource usage, and bottleneck information.

The full AES pipeline experiments use a CTR-style workload to study block-level throughput. The isolated `SubBytes + ShiftRows` workload processes 4096 independent AES states, corresponding to $4096 \times 16 = 65536$ bytes, or 64 KiB of data. The report analysis uses the following metrics:

- **scheduled II**, the local initiation interval of a loop
- **block estimated exit II**, the estimated number of cycles between completed loop iterations leaving the pipeline
- **pipeline latency**, inferred from invocation latency, trip count, and II
- **estimated throughput**, calculated from block size, exit II estimated clock frequency
- **resource usage**, including ALUTs, FFs, RAMs, and MLABs.

The latency convention follows the HLS pipeline model described by de Fine Licht et al. (Fine Licht et al. 2018). If L_{inv} is the invocation latency, L_{pipe} is the pipeline latency, II is the initiation interval, and N_{trip} is the trip count, then

$$L_{\text{inv}} = L_{\text{pipe}} + \text{II}(N_{\text{trip}} - 1).$$

Therefore, the pipeline latency can be inferred as

$$L_{\text{pipe}} = L_{\text{inv}} - \text{II}(N_{\text{trip}} - 1).$$

Throughput is then estimated from the AES block size and clock frequency. An AES block is 128 bits, or 16 bytes, so the bit throughput at clock frequency f_{max} is

$$T_{\text{bits}} = \frac{128 f_{\text{max}}}{\text{II}_{\text{exit}}}.$$

For reporting in MB/s, the equivalent expression is

$$T_{\text{MB/s}} = \frac{16 f_{\text{max}}}{\text{II}_{\text{exit}} \times 10^6}.$$

This report uses II_{exit} rather than only the scheduled II when estimating completed-block throughput. This is a conservative choice, but it avoids overstating designs where an inner

loop can start frequently while the top-level pipeline still releases completed blocks slowly. The analysis therefore treats latency, throughput, and resource use as related but distinct quantities.

4 Implementation and Experimental Setup

Two implementation tracks were maintained. The first was a full-round CTR-style AES pipeline integrating `SubBytes`, `ShiftRows`, `MixColumns`, and `AddRoundKey` in a single FPGA-oriented kernel flow. The second was an isolated `SubBytes` + `ShiftRows` kernel used as a controlled case study for detailed optimisation. This split was deliberate and to divide work within the team. The full-round kernel captures system-level interactions, while the isolated stage kernel makes it easier to attribute changes in II, latency, and resource use to specific source-level transformations.

The full-round implementation was explored through a sequence of system-level optimisations: deep pipelining of the AES round structure, S-box banking, decoupling and buffering of the keystream path, explicit non-aliasing constraints on pointer arguments, and loop-independence restructuring. These optimisations were chosen because they target the main barriers to a streaming AES datapath: long combinational chains, memory-port contention, conservative memory-aliasing assumptions, and loop-carried dependencies.

The isolated stage kernel was compiled in four variants. The naive baseline performed `SubBytes` over the 16-byte state and then applied `ShiftRows`. The unrolled variant attempted to expose byte-level parallelism directly. The direct fused variant merged substitution and row shifting into one loop to remove the temporary substituted-state array. The buffered fused variant loaded one AES state into local storage, performed the fused transform locally, and then wrote the output back to global memory. This final variant was designed to separate global memory access from computation, reducing the memory-dependency path seen in the direct fused design.

For each variant, the same reporting process was used. The HTML optimisation report supplied the loop-analysis values, while the resource summary supplied the ALUT, FF, RAM, and MLAB counts. Emulator timing was retained as a useful development metric, but final interpretation was based primarily on report metrics because they better describe the expected FPGA structure.

5 Full AES Pipeline Optimisation

The full AES pipeline was used to test how far the design could move from a serial round structure towards a streaming accelerator. This is the central challenge in FPGA AES design. Prior work reports large throughput gains when AES rounds are deeply pipelined and memory access is organised so that new blocks can enter the datapath before previous blocks have completed (Manjith and Ramasubramanian 2018; Soltani and Sharifian 2015). In the oneAPI flow, this same issue appears through the reported exit II: a high exit II means that the datapath may be doing useful work internally, but completed blocks are still leaving too slowly.

Table 1: Full AES pipeline optimisation sequence. Throughput is calculated using $T_{\text{MB/s}} = 16f_{\text{max}}/(\text{II}_{\text{exit}} 10^6)$ with $f_{\text{max}} = 480$ MHz.

Optimisation step	Latency	Exit II	Throughput	Interpretation
Naive full pipeline	24650	24658	0.31 MB/s	Effectively serial execution, little useful overlap between blocks.
Deep pipelining attempt	164	265	29 MB/s	Large reduction in latency, but dependencies still prevent a true streaming pipeline.
Memory banking	194	325	23 MB/s	Additional S-box banking did not help the system-level bottleneck.
Decouple and buffer	173	219	35 MB/s	Separating data movement from computation reduced the completion interval.
Guarantee non-aliasing	128	136	56 MB/s	Removing conservative pointer assumptions reduced false memory dependencies.
Loop independence	161	3	2600 MB/s	The compiler could finally treat successive blocks as mostly independent.

Table 1 shows that the naive full-round pipeline was almost entirely serial. Its latency and exit II were nearly identical: 24650 and 24658 cycles respectively. This indicates that a new block could not be completed until the previous block had effectively passed through the whole datapath. Such a design may be functionally correct, but it fails to use the FPGA as a throughput-oriented pipeline.

The deep-pipelining attempt produced the first major improvement. Latency fell from 24650 cycles to 164 cycles, and throughput increased from 0.31 MB/s to 29 MB/s. This confirms that separating the AES computation into clearer pipeline stages gives the compiler more opportunity to overlap work. However, the exit II remained 265, much larger than the block latency. This is a useful diagnostic result: the datapath had become shorter, but the compiler still could not safely accept or complete new blocks at a high rate.

The S-box banking experiment was intentionally aimed at one of the most obvious AES bottlenecks. Since `SubBytes` performs repeated lookup-table accesses, one might expect banking to reduce memory-port contention. In this experiment, however, memory banking reduced throughput from 29 MB/s to 23 MB/s. This does not mean S-box banking is ineffective, memory partitioning is a standard FPGA AES technique (Manjith and Ramasubramanian 2018). Instead, it shows that this particular design was not limited primarily by S-box port contention at that stage. The extra memory structure appears to have added overhead without removing the dominant dependency.

Decoupling and buffering improved the result more meaningfully, reducing exit II from 325 to 219. This is consistent with the oneAPI guidance that memory dependencies can limit loop pipelining when the compiler must preserve ordering between loads and stores (Intel 2023d). By separating keystream generation and data movement, the design reduced the pressure on a single loop-carried path. This result is also conceptually related to the buffered `SubBytes` + `ShiftRows` stage discussed later: in both cases, the improvement came from restructuring memory access rather than simply adding more parallel logic.

The largest full-pipeline gain came from explicitly exposing independence. Guaranteeing that plaintext, ciphertext, key, and counter pointers did not alias reduced exit II from 219 to 136. This matters because an HLS compiler must otherwise assume that two pointer arguments might refer to overlapping memory, which can force unnecessary ordering constraints. The final loop-independence transformation reduced exit II further to 3, increasing estimated throughput to 2600 MB/s. This aligns with Intel’s description of `ivdep`-style loop independence annotations: when used correctly, they tell the compiler that there are no relevant loop-carried memory dependencies and may reduce II (Intel 2023b). The warning is equally important: such annotations are only valid when the programmer can prove the dependency is false. In this case, the large throughput gain is best interpreted as evidence that the earlier pipeline was limited mainly by conservative dependency assumptions rather than by the intrinsic AES arithmetic.

6 SubBytes and ShiftRows Stage Optimisation

The isolated `SubBytes` + `ShiftRows` kernel provides a more controlled view of the same design problem. The naive version followed the AES specification directly: it first substituted all 16 bytes through the S-box and then applied the row permutation. Three modifications were tested. Inner-loop unrolling exposed byte-level parallelism directly. Direct fusion removed the temporary substituted-state array by combining substitution and permutation in a single loop. Buffered fusion kept the fused computation but changed the dataflow to load one state locally, compute on local storage, and then store the result.

Table 2: Isolated `SubBytes` + `ShiftRows` optimisation results. Emulator time is shown only as a development metric. The FPGA-oriented interpretation is based on exit II, resource use, and bottleneck reports.

Stage variant	Emu.	Exit II	Thr.	ALUTs	FFs	RAMs	Main observation
Naive baseline	1.19 ms	1126	6.82 MB/s	2805	6523	35	Clear baseline, but shallow top-level completion rate.
Inner loops unrolled	0.043 ms	69	111.30 MB/s	5901	11521	32	Faster emulator result, but much higher logic use and handshaking off.
Direct fused, no buffering	0.0385 ms	95	80.84 MB/s	1656	4927	10	Small area, but severe fused-loop memory dependency.
Buffered fused load-compute-store	0.0445 ms	1041	7.38 MB/s	2378	5675	26	Lower area than naive, the inner loops recovered to $II = 1$.

Table 2 demonstrates why emulator timing cannot be used as the main performance metric for this project. The unrolled and direct-fused variants were the fastest under `fpga_emu`, but both produced warning signs in the report. Loop unrolling is a standard FPGA optimisation because it can expose independent operations to the compiler (Intel 2023f). Here, however, unrolling reduced the top-level exit II from 1126 to 69 only by more than doubling ALUT use from 2805 to 5901 and increasing FF use from 6523 to 11521. Hyper-optimised handshaking also turned off. This is not automatically a failed design, since FPGA optimisation often spends more area to gain throughput. However, the resource increase was large for a small stage kernel, and the compiler still reported a memory dependency. The result is therefore better described as an expensive throughput trade-off than as a clean optimisation.

Direct fusion was the opposite failure mode. It reduced ALUTs to 1656, FFs to 4927, and RAMs to 10, making it the smallest stage design. However, the fused inner loop degraded to scheduled $II = 95$ because a global load, S-box lookup, and global store were placed on the same

loop-carried path. This is a classic HLS scheduling problem: the source code looks simpler, but the memory ordering constraints become harder for the compiler to pipeline (Intel 2023a; Intel 2023d). The result is valuable because it shows that reducing source-code structure or intermediate arrays is not automatically beneficial on FPGA.

The buffered fused design was the most balanced local result. Its top-level throughput remained close to the naive baseline because the stage kernel was still a shallow load-compute-store structure rather than a deeply streaming pipeline. This explains the apparently modest throughput of 7.38 MB/s.

The oneAPI report also provides a Schedule Viewer, which was used as a qualitative check on the numerical loop-analysis results. For the buffered fused variant, the viewer showed long load and store regions around the fused byte transformation, supporting the interpretation that the stage was limited mainly by load-compute-store orchestration. The quantitative latency and throughput values reported here were still calculated from the Loop Analysis fields, following the HLS latency relation in de Fine Licht et al. Fine Licht et al. 2018.

Nevertheless, it improved the stage implementation in the ways that matter for reintegration into the full AES pipeline: ALUTs fell by about 15%, FFs by about 13%, and RAM blocks by about 26% relative to the naive baseline, while the inner load and compute loops both recovered to scheduled $II = 1$ and exit $II = 1$. The key change was that global memory access was separated from local computation, matching the general HLS principle of moving loop-carried memory pressure into local or private storage where possible (Intel 2023e).

7 Results and Discussion

The full-pipeline and stage-level experiments point to the same core conclusion: AES acceleration on FPGA is limited less by the AES arithmetic itself than by whether the compiler can see independent work. The full pipeline achieved its largest improvement only after non-aliasing and loop-independence information was made explicit. The isolated stage study showed the same principle at smaller scale: direct fusion failed because it created a memory-dependency path, while buffered fusion improved the local implementation by separating memory movement from computation.

The results also clarify the role of area. A common intuition is that using more FPGA area should improve speed because more logic can operate in parallel. This is only partly true. The unrolled `SubBytes + ShiftRows` stage did use more area and produced a lower exit II than the naive baseline, but it also increased ALUTs by about 110% and FFs by about 77%. That may be acceptable inside a larger design if the stage is the dominant bottleneck, but it is not automatically efficient. Conversely, direct fusion used very little area but left an unhealthy memory dependency. The best design choice therefore depends on both throughput and resource cost, not either metric alone.

The difference between the full pipeline and the isolated stage is especially important. The best full-pipeline result reached an exit II of 3, corresponding to an estimated 2600 MB/s at 480 MHz. This happened because the compiler could treat successive AES blocks as independent and schedule them as a streaming pipeline. The buffered stage, by contrast, remained a small load-compute-store kernel. It was locally cleaner and cheaper, but it did not by itself create a deep block-level pipeline. This is why its isolated throughput was only 7.38 MB/s. The result is still useful: it produces a better stage building block, but the full-system throughput gain would only be realised after that stage is reintegrated into the deeply pipelined AES datapath.

These findings are consistent with published FPGA AES work, where high throughput is obtained by combining round-level pipelining, memory organisation, and high block concurrency (Manjith and Ramasubramanian 2018; Soltani and Sharifian 2015), while compact subpipelined designs show that resource use and throughput must be considered together (Li et al. 2023). The contribution of this project is to show the same set of issues through the oneAPI report workflow. The optimisation report was not a just a diagnostic tool as it directly shaped the design process by showing whether each change improved exit II, reduced false dependencies, or merely shifted the bottleneck elsewhere.

Figures 1 and 2 summarise the two main design stories. The full-pipeline diagram shows that the decisive gain came late in the optimisation sequence, when independence was made explicit. The stage-level diagram shows why buffering helped locally: global memory movement was separated from the fused byte transformation.

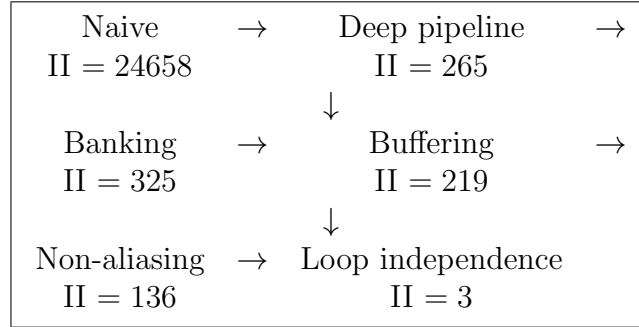


Figure 1: Full AES pipeline optimisation path, annotated with exit II.

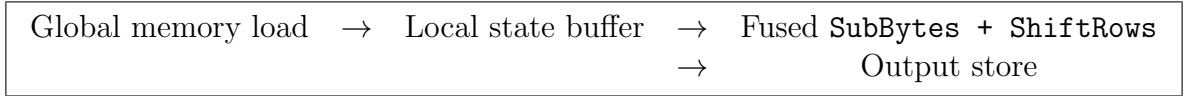


Figure 2: Buffered fused **SubBytes** + **ShiftRows** dataflow.

8 Limitations and Future Work

The main limitation is that the results are based primarily on compiler report estimates rather than board-level measurements. This is appropriate for early design exploration, but a final accelerator claim would require a successful Quartus hardware build and execution on the target FPGA. A hardware compile was attempted on the available FGPA's at IMPERIAL, but the build flow did not successfully trigger a full Quartus FPGA compilation, so reliable board-level timing and post-fit resource measurements could not be obtained. The current full-round design should therefore be treated as a structural AES acceleration study rather than a fully functional production AES-CTR implementation. A deployable core would require complete key expansion integration, known-answer-vector testing, successful hardware compilation, and timing validation on the target board.

The most immediate next step is to reintegrate the buffered fused **SubBytes** + **ShiftRows** stage into the best full-pipeline design. This would test whether the local area and scheduling improvements survive inside the streaming AES datapath. A second useful extension is automated report extraction: the oneAPI reports contain the required II, latency, throughput, and resource data in structured files, so a small parser could reduce manual transcription errors and make it easier to track more design variants.

9 Conclusion

This report investigated AES acceleration using Intel oneAPI SYCL and FPGA compiler reports. At full-pipeline level, the strongest result was obtained by making block independence visible to the compiler: the optimisation sequence reduced exit II from 24658 to 3, increasing estimated throughput from 0.31 MB/s to 2600 MB/s at 480 MHz. This shows that the main barrier was not the AES round logic alone, but the compiler’s ability to prove that consecutive blocks could be processed independently.

At stage level, the **SubBytes + ShiftRows** study showed why local optimisation must be interpreted carefully. Unrolling improved the derived throughput but used much more area, direct fusion reduced area but created a memory-dependency bottleneck, and buffered fusion gave the best local balance by reducing resources while restoring healthy inner-loop scheduling. The overall result is that effective FPGA AES acceleration depends on exposing true parallelism, controlling memory dependencies, and using the optimisation report as a design guide rather than relying on emulator timing alone.

AI Use Declaration

Portions of this document were assisted by AI tools for drafting and editing. Further uses include assistance in understanding some technical details within academic papers and LaTeX formatting.

References

- AbdAllah, Eslam G., Yu Rang Kuang, and Changcheng Huang (2020). “Advanced Encryption Standard New Instructions (AES-NI) Analysis: Security, Performance, and Power Consumption”. In: *Proceedings of the 2020 12th International Conference on Computer and Automation Engineering (ICCAE)*. Sydney, NSW, Australia: Association for Computing Machinery. DOI: [10.1145/3384613.3384648](https://doi.org/10.1145/3384613.3384648).
- Dworkin, Morris (2001). *Recommendation for Block Cipher Modes of Operation: Methods and Techniques*. Tech. rep. Special Publication 800-38A. National Institute of Standards and Technology. DOI: [10.6028/NIST.SP.800-38A](https://doi.org/10.6028/NIST.SP.800-38A).
- Fine Licht, Johannes de et al. (2018). *Transformations of High-Level Synthesis Codes for High-Performance Computing*. DOI: [10.48550/arXiv.1805.08288](https://doi.org/10.48550/arXiv.1805.08288).
- Intel (2023a). *Fuse Loops to Reduce Overhead and Improve Performance*. <https://www.intel.com/content/www/us/en/docs/oneapi-fpga-add-on/optimization-guide/2023-2/fuse-loops.html>.
- (2023b). *ivdep Attribute*. <https://www.intel.com/content/www/us/en/docs/oneapi-fpga-add-on/optimization-guide/2023-1/ivdep-attribute.html>.
- (2023c). *Loops*. <https://www.intel.com/content/www/us/en/docs/oneapi-fpga-add-on/optimization-guide/2023-2/loops.html>.

- Intel (2023d). *Minimize the Memory Dependencies for Loop Pipelining*. <https://www.intel.com/content/www/us/en/docs/oneapi-fpga-add-on/optimization-guide/2023-1/min-mem-dep.html>.
- (2023e). *Transfer Loop-Carried Dependency to Local Memory*. <https://www.intel.com/content/www/us/en/docs/oneapi-fpga-add-on/optimization-guide/2023-2/transfer-loop-carried-dependency-to-local-memory.html>.
 - (2023f). *unroll Pragma*. <https://www.intel.com/content/www/us/en/docs/oneapi-fpga-add-on/optimization-guide/2023-2/unroll-pragma.html>.
 - (2024a). *Intel oneAPI DPC++/C++ Compiler Handbook for FPGAs Overview*. <https://www.intel.com/content/www/us/en/docs/oneapi-fpga-add-on/developer-guide/2024-2/fpga-handbook.html>.
 - (2024b). *Single Work-item Kernels*. <https://www.intel.com/content/www/us/en/docs/oneapi-fpga-add-on/developer-guide/2024-2/single-work-item-kernels.html>.
- Li, Ke, Hua Li, and Graeme Mund (2023). “A reconfigurable and compact subpipelined architecture for AES encryption and decryption”. In: *EURASIP Journal on Advances in Signal Processing* 2023.1. DOI: [10.1186/s13634-022-00963-3](https://doi.org/10.1186/s13634-022-00963-3).
- Manjith, Baby Chellam and Natarajan Ramasubramanian (2018). “AES Hardware Accelerator on FPGA with Improved Throughput and Resource Efficiency”. In: *Arabian Journal for Science and Engineering* 43.12, pp. 6873–6890. DOI: [10.1007/s13369-017-2925-0](https://doi.org/10.1007/s13369-017-2925-0).
- National Institute of Standards and Technology (2001). *Advanced Encryption Standard (AES)*. Tech. rep. FIPS PUB 197. NIST. DOI: [10.6028/NIST.FIPS.197](https://doi.org/10.6028/NIST.FIPS.197).
- Soltani, Abolfazl and Saeed Sharifian (2015). “An ultra-high throughput and fully pipelined implementation of AES algorithm on FPGA”. In: *Microprocessors and Microsystems* 39.7, pp. 480–493. DOI: [10.1016/j.micpro.2015.07.005](https://doi.org/10.1016/j.micpro.2015.07.005).